

Федеральное государственное автономное образовательное учреждение
высшего
образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчет

По лабораторной работе №2

По дисциплине «Основы схемотехники»

Вариант 6.

Выполнили: Матевосян Артур Русланович и
Антипин Григорий , группа Р3332
Преподаватель: Табунщик С. М.

Санкт-Петербург

2025

Оглавление

Цель работы 3

Задание	3
Отчет о проделанной работе	3
Область допустимых значений.....	3
Умножитель 16x8	4
Функция $ab+a^3$	4
Описание работы	6
Тестовое окружение	8
Результат тестирования	9
Выводы.....	12

Цель работы

Получить навыки описания арифметических блоков на RTL-уровне с использованием языка описания аппаратуры Verilog HDL.

Задание

6	$y = a \cdot b + a^3$	2 сумматора и 1 умножитель
---	-----------------------	----------------------------

1. Разработайте и опишите на Verilog HDL схему, вычисляющую значение функции в соответствии с заданными ограничениями согласно варианту задания.
2. Определите область допустимых значений функции.
3. Разработайте тестовое окружение для разработанной схемы. Тестовое окружение должно проверять работу схемы не менее, чем на 10 различных тестовых векторах.
4. Проведите моделирование работы схемы и определите время вычисления результата. Схема должна тактироваться от сигнала с частотой 100 МГц.
5. Составьте отчет по результатам выполнения работы.

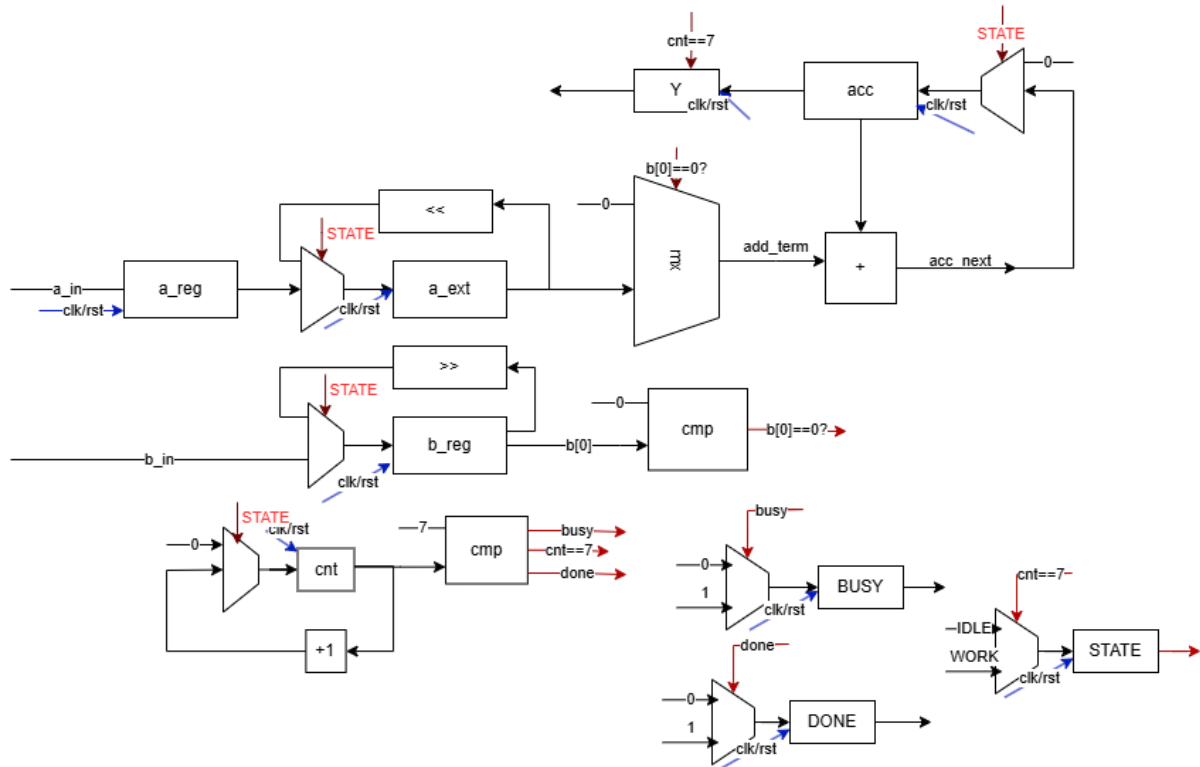
Отчет о проделанной работе

Область допустимых значений

8-битные входные значения – [0; 255]

24-битное выходное значение [0; 16646400]

Умножитель 16х8



Описание работы.

Подается стартовый сигнал, переход в состояние WORK. Защелкиваем значения на входах в `a_ext` (расширяя битность до 24) и `b_reg`, обнуляем регистры аккумулятора. В состоянии WORK на каждом такте смотрим на 0 бит множителя `b_reg`, если он не равен нулю, то прибавляем к асс значение `a_ext`. `a_ext` сдвигаем влево на 1 бит, `b_reg` вправо на 1 бит. Увеличиваем `cnt` на 1. Если `cnt==7`, то умножение завершено, в регистр `Y` попадает результат умножения, `BUSY`=0, `DONE`=1, `STATE`=IDLE.

// mult.v — последовательный умножитель "в столбик" (16×8 бит)

```
module mult #(parameter W1 = 16, parameter W2 = 8)(
    input clk,
    input rst,
    input start,
    input [W1-1:0] a,
    input [W2-1:0] b,
    output reg [W1+W2-1:0] y,
    output reg done
);
    reg [W1-1:0] A;
    reg [W2-1:0] B;
    reg [W1+W2-1:0] acc;
```

```

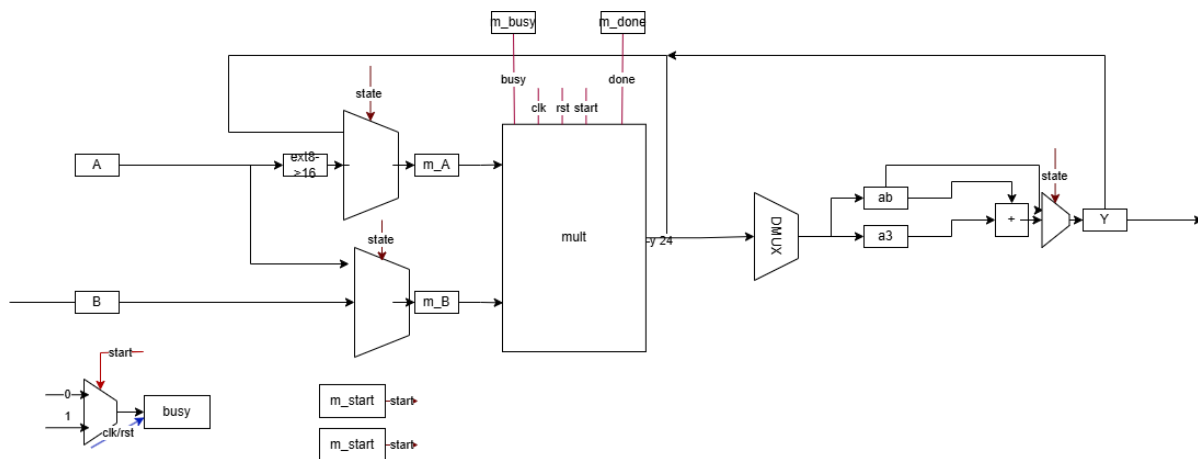
reg [3:0] count;
reg busy;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        y <= 0;
        acc <= 0;
        A <= 0;
        B <= 0;
        count <= 0;
        done <= 0;
        busy <= 0;
    end else begin
        done <= 0;
        if (start && !busy) begin
            A <= a;
            B <= b;
            acc <= 0;
            count <= 0;
            busy <= 1;
        end else if (busy) begin
            if (B[0])
                acc <= acc + A;
            A <= A << 1;
            B <= B >> 1;
            count <= count + 1;

            if (count == W2-1) begin
                y <= acc;
                done <= 1;
                busy <= 0;
            end
        end
    end
end
end
endmodule

```

Функция $ab+a^3$



Описание работы

// func.v — модуль вычисления функции $y = a*b + a^3$

```
module func #(parameter W = 8) (
    input clk,
    input rst,
    input start,
    input [W-1:0] a,
    input [W-1:0] b,
    output reg [2*W-1:0] y,
    output reg done,
    output reg busy
);

    // --- состояния автомата ---
    localparam IDLE = 3'd0;
    localparam MUL1 = 3'd1; // a*b
    localparam MUL2 = 3'd2; // a*a
    localparam MUL3 = 3'd3; // (a^2)*a
    localparam ADD = 3'd4; // сложение
    localparam DONE = 3'd5;

    reg [2:0] state;

    // --- сигналы к умножителю ---
    reg start_mult;
    reg [W-1:0] opA, opB;
    wire [23:0] mult_out; // 16*8 = 24 бита
    wire done_mult;
```

```

// --- промежуточные результаты ---
reg [15:0] ab_res;    // a*b
reg [15:0] a2_res;    // a^2
reg [23:0] a3_res;    // a^3 (шире!)

// --- итерационный умножитель (16×8 бит) ---
mult #(16, 8) u_mult(
    .clk(clk),
    .rst(rst),
    .start(start_mult),
    .a({8'b0, opA}), // расширяем opA до 16 бит
    .b(opB),
    .y(mult_out),
    .done(done_mult)
);

// --- автомат управления ---
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state    <= IDLE;
        done     <= 0;
        busy     <= 0;
        y        <= 0;
        ab_res    <= 0;
        a2_res    <= 0;
        a3_res    <= 0;
        start_mult <= 0;
        opA       <= 0;
        opB       <= 0;
    end else begin
        start_mult <= 0; // по умолчанию не дёргаем
        done       <= 0;

        case (state)
            IDLE: begin
                if (start) begin
                    busy    <= 1;
                    opA     <= a;
                    opB     <= b;
                end
            end
        endcase
    end
end

```

```

        start_mult <= 1; // запускаем a*b
        state    <= MUL1;
    end
end

MUL1: begin
    if (done_mult) begin
        ab_res    <= mult_out[15:0]; // сохраняем a*b
        opA       <= a;
        opB       <= a;
        start_mult <= 1;           // запускаем a*a
        state     <= MUL2;
    end
end

MUL2: begin
    if (done_mult) begin
        a2_res    <= mult_out[15:0]; // сохраняем a^2 полностью
        opA       <= mult_out[15:0]; // теперь не обрезаем!
        opB       <= a;
        start_mult <= 1;           // запускаем (a^2)*a
        state     <= MUL3;
    end
end

MUL3: begin
    if (done_mult) begin
        a3_res <= mult_out; // сохраняем a^3 (24 бита)
        state <= ADD;
    end
end

ADD: begin
    y    <= ab_res + a3_res[15:0]; // складываем
    state <= DONE;
end

DONE: begin
    done <= 1;
end

```



```

        busy <= 0;
        state <= IDLE;
    end
endcase
end
end
endmodule

```

Тестовое окружение

```

// func_tb.v — тестбенч для проверки работы func
`timescale 1ns/1ps

module func_tb;
    reg clk, rst, start;
    reg [7:0] a, b;
    wire [15:0] y;
    wire done, busy;

    func #(8) dut (
        .clk(clk), .rst(rst), .start(start),
        .a(a), .b(b), .y(y), .done(done), .busy(busy)
    );

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, func_tb);

        clk = 0;
        rst = 1;
        start = 0;
        a = 0; b = 0;
        #10 rst = 0;

        // Тест 1
        @(negedge clk);
        a = 3; b = 4;
        start = 1;
        @(negedge clk);
        start = 0;
    end
endmodule

```

```
wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 39)", a, b, y);

// Тест 2
@(negedge clk);
a = 5; b = 2;
start = 1;
@(negedge clk);
start = 0;

wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 135)", a, b, y);

// Тест 3
@(negedge clk);
a = 10; b = 10;
start = 1;
@(negedge clk);
start = 0;

wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 1100)", a, b, y);

// Дополнительные тесты
@(negedge clk); a = 0; b = 0; start = 1; @(negedge clk); start = 0; wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 0)", a, b, y);

@(negedge clk); a = 1; b = 5; start = 1; @(negedge clk); start = 0; wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 6)", a, b, y);

@(negedge clk); a = 2; b = 7; start = 1; @(negedge clk); start = 0; wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 22)", a, b, y);

@(negedge clk); a = 4; b = 3; start = 1; @(negedge clk); start = 0; wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 76)", a, b, y);

@(negedge clk); a = 8; b = 1; start = 1; @(negedge clk); start = 0; wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 520)", a, b, y);
```

```

@(negedge clk); a = 10; b = 0; start = 1; @(negedge clk); start = 0; wait(done);
#1 #1$display("a=%0d b=%0d y=%0d (ожидалось 1000)", a, b, y);

@(negedge clk); a = 15; b = 15; start = 1; @(negedge clk); start = 0; wait(done);
#1 $display("a=%0d b=%0d y=%0d (ожидалось 3600)", a, b, y);

    #20 $finish;
end

always #5 clk = ~clk;
endmodule

```

Результат тестирования

```

VCD info: dumpfile dump.vcd opened for output.
a=3 b=4 y=39 (ожидалось 39)
a=5 b=2 y=135 (ожидалось 135)
a=10 b=10 y=1100 (ожидалось 1100)
a=0 b=0 y=0 (ожидалось 0)
a=1 b=5 y=6 (ожидалось 6)
a=2 b=7 y=22 (ожидалось 22)
a=4 b=3 y=76 (ожидалось 76)
a=8 b=1 y=520 (ожидалось 520)
a=10 b=0 y=1000 (ожидалось 1000)
a=15 b=15 y=3600 (ожидалось 3600)
func_tb.v:77: $finish called at 3326000 (1ps)

```



Выводы

В ходе работы был разработан аппаратный блок вычисления функции $y = a \cdot b + a^3$ на языке **Verilog HDL**.

Схема включает итерационный умножитель и управляющий автомат, который последовательно выполняет три операции умножения и одно сложение.

Были изучены принципы синхронной логики, взаимодействие модулей по сигналам **clk**, **rst**, **start**, **done**, а также построена структурная схема устройства.

Цель работы — реализация и исследование аппаратного блока вычисления функции — успешно достигнута.